



How Do Matrices and Adjacency Lists Work?

Graphs are very powerful data structures made by a set of nodes, also known as vertices, and edges that connect them.

There are two common ways to implement graphs in your code:

- Adjacency matrices
- Adjacency lists

Let's look more into these, and go over their advantages and limitations.

Adjacency Matrices

We'll start with adjacency matrices.

An adjacency matrix is a two-dimensional list in which the rows and columns represent the graph's vertices.

The values in the matrix represent the edges or connections between the nodes.

For example, if you have a matrix stored in a variable named `matrix`, the value stored at `matrix[i][j]`, where `i` is the row and `j` is the column, represents the edge or connection between node `i` and node `j`.

The values may have different meanings depending on whether the graph is weighted or unweighted:

- If the graph is unweighted, a value of `1` means that there is an edge connecting these nodes, while a value of `0` means there is no edge between them.
- If the graph is weighted, the value would represent the weight of the edge that connects the nodes.

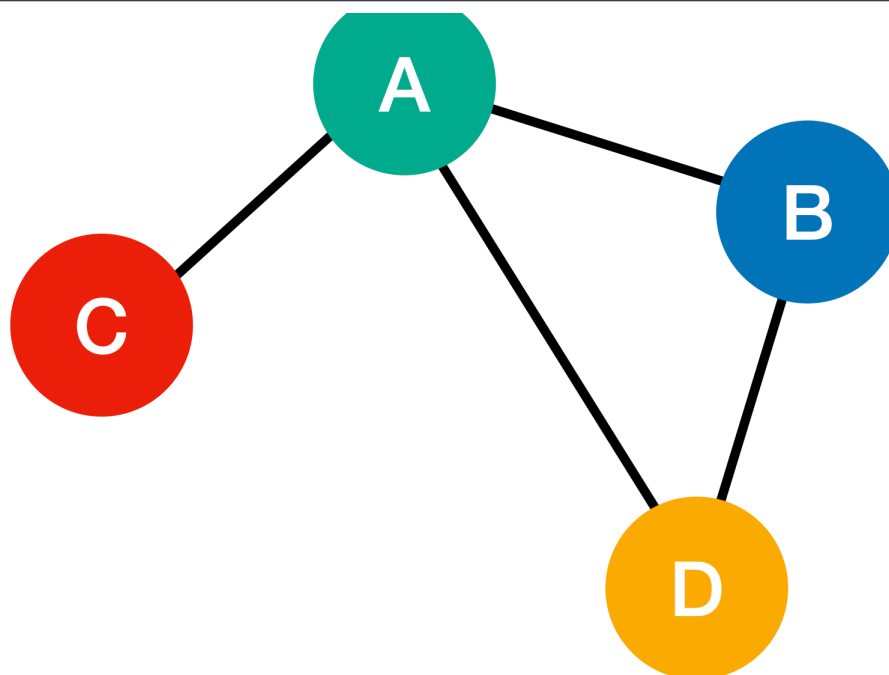
One of the great advantages of using an adjacency matrix is that checking if there is an edge between two nodes takes constant time $O(1)$. This is because the program only needs to find that particular value in the 2D list.

However, this efficiency in finding the edges comes with a tradeoff. Adjacency matrices have a large quadratic space requirement $O(V^2)$, where `V` is the number of nodes in the graph.

This is inefficient for sparse graphs, which are graphs that only have a few edges. Why? Because if the graph is sparse, you will be storing many 0s in the matrix to represent the lack of edges between the nodes, and these 0s will still take space in memory.

Adjacency matrices are also inefficient for finding a node's neighbors because the program has to iterate over the entire row or column to find the 0s and 1s that represent the edges. In the worst case, this process can take $O(V)$ time, where `V` is the number of nodes in the graph.

Let's see an example of an adjacency matrix for this particular graph:



In the adjacency matrix:

- Each row represents a node. The first row represents node A, the second row represents node B, and so on.
- Each column represents a node too.
- Each value in the matrix represents whether there is an edge between each pair of nodes. A value of 0 means there isn't an edge connecting these nodes, while a value of 1 means there is an edge.

The values in the diagonal represent whether or not each node has a self-loop, an edge that connects the node to itself. In our example, they are all 0s because the graph doesn't have any self-loops.

This is a visual representation of the adjacency matrix to show you how the rows and columns represent the corresponding nodes.

For example, the first row is `[0, 1, 1, 1]` because node A has edges connecting it to nodes B, C, and D:

```
#      A  B  C  D
# A    [0, 1, 1, 1],
# B    [1, 0, 0, 1],
# C    [1, 0, 0, 0],
# D    [1, 1, 0, 0]
```

And this is the same adjacency matrix, but implemented in Python code:

```
adjacency_matrix = [
    [0, 1, 1, 1], # The neighbors of A are B, C, and D
    [1, 0, 0, 1], # The neighbors of B are A and D
    [1, 0, 0, 0], # The only neighbor of C is A
    [1, 1, 0, 0]  # The neighbors of D are A and B
]
```

Adjacency Lists



There are two ways to implement adjacency lists:

- As an array, where each index represents a node and the list stored at that index contains its neighbors.
- As a dictionary, where each key represents a node and the value associated to that key (a list) contains its neighbors.

Adjacency lists are more efficient than adjacency matrices in terms of space requirements. They have a $O(V + E)$ space complexity, where V is the number of vertices (nodes) and E is the number of edges.

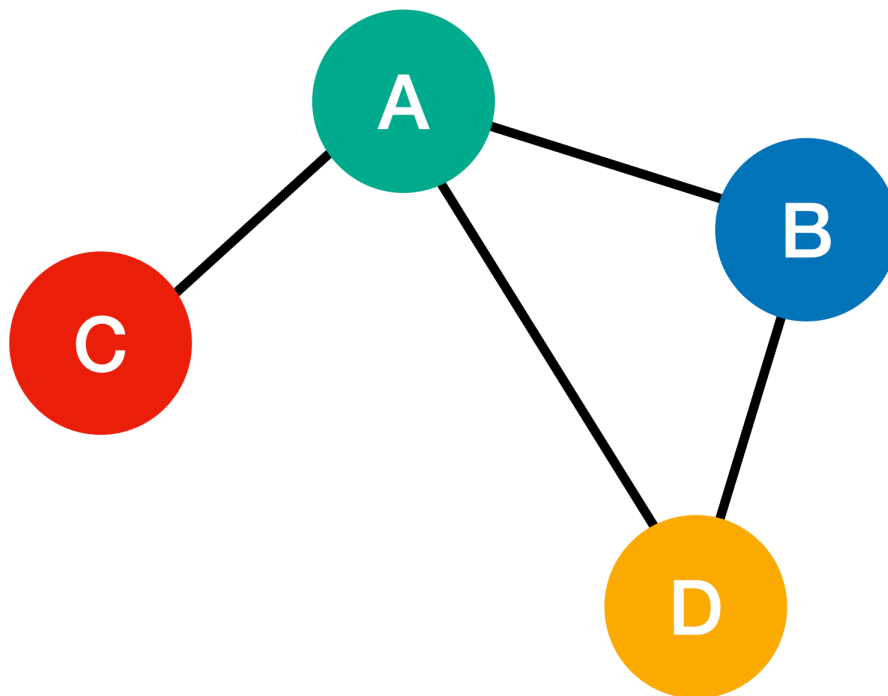
It's also efficient for finding all neighbor nodes, since this operation only requires accessing the list associated to the node.

However, there is also a tradeoff.

Adjacency lists are less efficient than adjacency matrices for determining if there is an edge between two nodes.

The search process can take $O(V)$ time in the worst-case, since it may have to search through a very long list of neighbors if the node is connected to all the other nodes in the graph.

Here is an example of an adjacency list for this graph:



This adjacency list is implemented as a dictionary. Every key in the dictionary represents a node, and the value associated to that key is a list with all the neighbors of the corresponding node:

```
adjacency_list = {  
    'A': ['B', 'C', 'D'],  
    'B': ['A', 'D'],  
    'C': ['A'],  
    'D': ['A', 'B']  
}
```



```
adjacency_list = [  
    ['B', 'C', 'D'], # Neighbors of A (index 0)  
    ['A', 'D'],     # Neighbors of B (index 1)  
    ['A'],          # Neighbors of C (index 2)  
    ['A', 'B']      # Neighbors of D (index 3)  
]
```

Notice that even if this 2D list may look similar to the adjacency matrix, they are quite different.

- The adjacency matrix stores 0s, 1s, or other values that represent the edges or weights of the edges in the graph.
- The adjacency list stores the actual list of all the neighbors of each node.

This is a very important difference that you should be familiar with.

Both adjacency matrices and adjacency lists are very important for implementing graphs. Choosing between them depends on the graph's size and how you need to use the data. Adjacency matrices are helpful for dense graphs with many edges, while adjacency lists are usually the preferred choice for real-world scenarios, where sparse graphs are more common.

Questions

For which of the following scenarios is an adjacency matrix the most efficient choice for representing a graph?

A social network with billions of people and very few connections per person.

A computer network with only five connections.

A dense graph where every node is connected to most other nodes.

Correct!

A graph where the main operation is to find all neighbors of a specific node.

When would it be more efficient to use an adjacency list over an adjacency matrix?

When the graph is dense and has many edges.

When you need to check if an edge exists between two nodes very quickly.

When the graph has a high number of vertices and many connections.

When the graph is sparse, with a high number of vertices and a low number of edges.

Correct!



Python Certification



Understanding Graphs and Trees

Finding all neighbors of a single node.

Checking if a direct edge exists between two specific nodes.

Correct!

Iterating through all nodes in the graph.

Adding a new node to the graph.

Check your answer

Ask for Help